Carácter:

- $int\ a\ =\ 'z'$; // Almacena el valor en código ASCII de “z” [minúscula].
- $char\ c\ =\ 'z'$; // Almacena la letra “z” [minúscula].

Cadenas: Series de caracteres tratados como una unidad.

- $char\ c[\ ]\ =\ "Algoritmos"$; // Almacena la palabra **“Algoritmos”** y le define un espacio de 11 espacios a la cadena c, que corresponden a las 10 letras de “Algoritmos” y al nulo “\n”.
- El valor o palabra inicial de una cadena *solamente se puede inicializar en la línea de declaración*. Luego no se puede hacer, por ejemplo: $c\ =\ "Ritmosalgo"$; // Ya que tiraría error.
- $char\ color[\ ]\ =\ "azul"$; es distinto de → $char\ color[\ ]\ =\ \{'a',\ 'z',\ 'u',\ 'l'\}$; // *La primera coloca automáticamente el ‘\n’, mientras que la segunda no*, por lo tanto no nos dejará utilizar las funciones y librerías para manejo de cadenas.
- En **todas** las cadenas de caracteres se agrega el nulo al final, por lo tanto hay que tener en cuenta de que el *espacio para almacenar* sea → (longitud de la palabra + 1 {espacio para el nulo}).



- $char\ *\ colorPtr\ =\ "azul"$; // Podemos declarar una cadena como un *puntero tipo char **. El puntero apunta al primer carácter, es decir, almacena la dirección de memoria del primer carácter de “azul”.

Strings:

- Cadenas más sofisticadas
- No tienen longitud máxima ya que tienen una gestión automática de la memoria (*son dinámicas*).
- Se operan más fácil que las cadenas de caracteres.

Entrada de cadenas:

1. El operador de entrada (**>>**) se comporta de la siguiente manera:
    a. *Elimina los espacios en blanco que hubiera al principio* de la entrada de datos.
    b. *Lee dicha entrada hasta que encuentre algún carácter de espacio en blanco*, el cual **no será leído** y **permanecerá en el buffer** de entrada hasta la próxima operación de entrada.
        i. Para evitar tener problemas con este tipo de ingresos y el buffer podemos usar $cin\ >>\ ws$; o $cin.ignore(1000.\ '\n')$; y limpiarlo.

Salida de cadenas:

- $cout\ <<\ cadena\ <<\ endl$; // Muestra por pantalla el contenido de una cadena.
- $cout\ <<\ cadena[i]\ <<\ endl$; // Muestra por pantalla el carácter almacenado en la iésima posición de *cadena*.

Librería <cstdio>

- $getchar()$; // Lee el próximo carácter.
  - $a = getchar()$; → Lee el siguiente carácter y almacena su valor en ASCII en a.
  - $c = getchar()$; → Lee el siguiente carácter y almacena 'carácter' en c.
- $gets(cadena)$; // Lee caracteres desde la entrada estándar hasta un carácter de *newline* o *EOF*. Agrega al final '\n'. Borra los carácteres anteriormente almacenados. *No funciona con strings.*
- $putchar(c)$; // Imprime el carácter almacenado en c sin un salto de línea, *Funciona con strings.*
- $puts(c)$; // Imprime la cadena c seguido de un endl. *No funciona con strings.*

Librería <cstring> { *Operaciones para trabajar sobre cadenas que tienen el carácter nulo '\n'* }
Ninguna de estas funciones opera con strings.

- $strcpy(c_dest,\ c_origen)$; // Supongamos que c_dest[n] y c_origen[m]. Mueve los m caracteres de c_origen a c_dest sin importar si m > n, es decir, hay que tener cuidado porque puedo mandar más valores de los que c_dest puede almacenar.
- $strncpy(c_dest,\ c_origen,\ lim)$; // Igual que el anterior solo que se limita hasta *lim*.
- $strcat(c_dest,\ c_origen)$; // Agrega al final de c_dest la cadena c_origen.
- $strncat(c_dest,\ c_origen,\ lim)$; // Igual que la anterior pero como mucho se agregan *lim* caracteres.
- $strlen(cad)$; // Devuelve la cantidad de caracteres de la cadena cad como entero.
- $strcmp(cad1,\ cad2)$; // Devuelve 0 si cad1==cad2, -1 si cad1>cad2 y 1 si cad2>cad1.

Librería <cctype> { *Proporciona principalmente operaciones sobre los valores tipo char* }

- $isalpha(c)$; // Devuelve 2 si c es minúscula, 1 si es mayúscula y 0 si no es una letra.
- $isalnum(c)$; // Devuelve 2 si es una minúscula, 1 si es una mayúscula, 0 si no es alfanumérico y 4 si es un número.
- $isupper(c)$; // Devuelve 1 si es una mayúscula y 0 si no (ya sea letra o cualquier carácter).
- $islower(c)$; // Devuelve 2 si es una minúscula y 0 si no.
- $isdigit(c)$; // Devuelve 1 si es un dígito y 0 si no.
- $isspace(c)$; // Devuelve 8 si es un espacio y 0 si no.
- $toupper(c)$; | $tolower(c)$; // *Transforma* el carácter en una mayúscula o minúscula *pero no lo modifica*. Es decir, si querés hacer que en una string sean todas mayúsculas por ej, para cada valor tenés que string[i]=*tolower/toupper(string[i]);*

Cadenas de caracteres: el tipo String <string>

- La librería <string> proporciona el **tipo string** para representar **cadena de caracteres**.
- Dispone de *operadores predefinidos* que permiten manejar cadenas de caracteres de forma muy simple e intuitiva.
- El tipo String puede ser utilizado para definir **constantes simbólicas, variables o parámetros formales** en los subprogramas.
  - Sin embargo, no es posible usar datos tipo string como elementos (campos) de un struct en **Archivos**.
- Los strings de C++ hacen *crecer el espacio de almacenamiento* para acomodarse a los cambios de tamaño de los datos de la cadena por encima de los límites de la memoria asignada inicialmente.

Tipo String: Entrada de cadenas.

- $cin >> string\&$; // Lee hasta que encuentra un espacio en blanco.
- $getline(cin, string\&)$; // Lee hasta que encuentra un '\n'
- $getline(cin, string\&, delimitador)$; // El delimitador por defecto es '\n' pero si se especifica un delimitador lee y almacena todos los caracteres del buffer hasta que encuentra el delimitador (no lo incluye).
- Es posible armar un arreglo de strings // $string\ nombre[cantidadElementos]$;

Problemas con el *getline:* Cuando se utiliza una lectura con getline después de una lectura previa con *cin >>* obtenemos un comportamiento que no corresponde con el esperado.

Solución: *eliminar los caracteres de espacios en blanco (y '\n') del buffer de entrada*. De esta forma el buffer descartará esos valores que dejó el *cin >>* y conseguiremos una ejecución correcta del *getline*.

Métodos para limpiar el buffer:

- $cin >> ws$; // Uso del manipulador ws en el flujo cin, que extrae todos los espacios en blanco hasta encontrar algún carácter distinto.
- $cin.ignore(a, b)$; // Elimina todos los caracteres del buffer de entrada en el flujo especificado, hasta que se haya eliminado el número de caracteres indicado en **a** o bien se haya eliminado el carácter indicado en **b**.
- La sentencia *cin >> ws;* se asocia a la función *getline* que le sigue, mientras que la sentencia *cin.ignore(a,b);* se asocia a la sentencia de entrada >> que aparece antes.

Operaciones con tipo String.

- Comparaciones lexicográficas con *operadores relacionales* (==, !=, >, <, etc)
  - if(string&>=autor){ /.../ }
- Concatenación de cadenas (+)

```
const string AUTOR = "José Luis" ;
int main ()
{
    string nombre = AUTOR + "López" ;
    nombre += "Vázque" ;
    nombre += 'z' ;
    nombre = AUTOR + 's' ;
```

- Longitud de la cadena. (size() | length()) || Devuelve como entero la longitud.
  - string&.size(); | string&.length();

Acceso a los caracteres de una cadena.

- Usando índices: string&[índice];
- Mediante función at(índice): string&.at(índice);

Operaciones con tipo String.

- $substr(posición, longitud)$; // Obtiene una *subcadena* a partir de la posición.
  - string cad = "abcdefg";
  - cout << cad.substr(2,3); // Muestra **cde**
- $find(subcadena)$; // Devuelve la *posición de la primera ocurrencia de la subcadena* en la cadena.
  - string cad = "Olala";
  - cout << cad.find("la"); // Muestra 1

- $rfind(subcadena)$; // Devuelve la *posición de la última ocurrencia de la subcadena* en la cadena.
    - string cad = “Olala”;
    - cout << cad.find(“la”); // Muestra 3
- $erase(posición,\ cantidad)$; // Elimina *cantidad* de caracteres a partir de *posición*. Si no se especifica cantidad borra hasta el final de la cadena.
- $insert(posición,\ subcadena)$; // Inserta la subcadena a partir de *posición*.
    - string cad = “abcdefgh”;
    - cad.insert(3,”123”); // cad ahora contiene “abc123defgh”